

13. Une variable, un octet

Retenir un nombre, et le montrer.

1. Ce qu'il faut comprendre

Une variable s'écrit `uint8_t` : un entier non signé de huit bits. C'est le seul nombre que cette machine connaisse — de 0 à 255, pas de virgule, pas de négatif. `int`, `char` et `auto` sont acceptés, et désignent la même chose ; `uint16_t`, `long` ou `float` sont **refusés avec leur numéro de ligne**, plutôt que traduits en douce sur un octet.

`attente++` est la façon C++ d'écrire `attente = attente + 1`. `+=`, `--`, `%=` existent aussi.

Pour afficher un chiffre calculé, on ne peut pas se servir de `texte()`, qui ne prend que des mots écrits en clair. On pose la **tuile** du chiffre : celle du 0 porte le numéro 27, et les neuf suivantes se suivent. D'où `27 + compte`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **929 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         64 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
"COMPTE"       6 o ← texte() — un texte gravé
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1      12 o sur place, dans main()
texte() × 1      11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte> // écrit un texte à l'écran
#include <poser> // pose une tuile sur une case du fond
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t compte = 0;
    uint8_t attente = 0;

    texte(4, 4, "COMPTE");

    while (true) {
        image();

        attente++;

        if (attente > 30) {
            attente = 0;

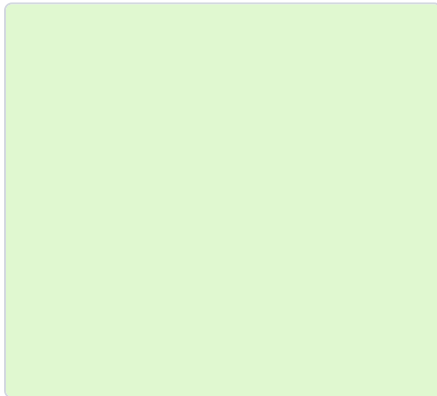
            if (compte < 9) compte++;
        }

        poser(11, 4, 27 + compte);
    }

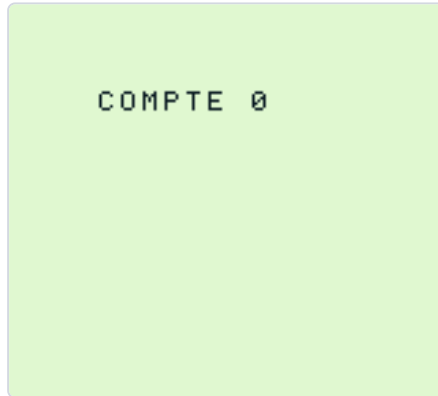
    return 0;
}
```

3. Ce qui se passe, étape par étape

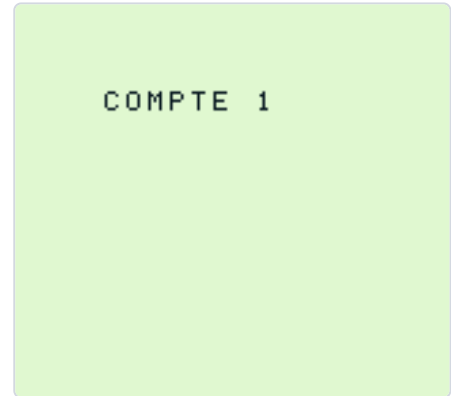
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



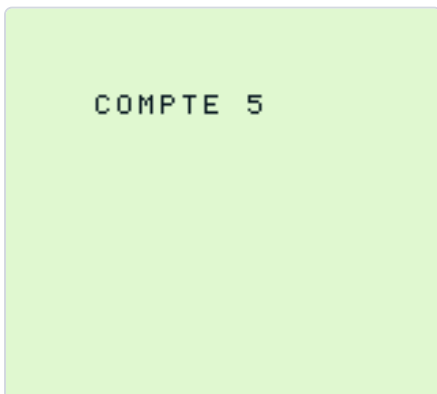
la toute première image — la console vient de démarrer — image 1



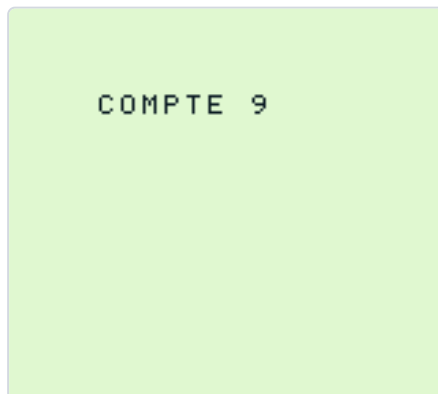
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un chiffre qui monte de 0 à 9, une fois par demi-seconde.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le compte part de zéro
- ✓ il monte tout seul
- ✓ il s'arrête à neuf
- ✓ et le chiffre est à l'écran

Ensuite : **14. Lire un bouton** — chapitre 3 : Retenir, et réagir. · `gameboy3` — écrire une cartouche Game Boy en C++